# RuralConnect

## A Multi-Role E-Commerce Marketplace for Rural Artisans

**Project Progress & Technical Implementation Report**

**Phase II: Monetization, Analytics, Governance & Direct Commerce**

*Authors:*
Kiran Pallathu
Yaswanth

*Domain:*
Web Development
Django / Python / PostgreSQL

**Reporting Period:** November 02, 2026 – November 07, 2026

November 9, 2026

# Contents

**1 Introduction & Week 2 Objectives** **1**
1.1 Phase Review & Evolution . . . . . 1
1.2 Primary Objectives for Week 2 . . . . . 1

**2 System Architecture & Module Additions** **2**
2.1 Modular Architecture Progression . . . . . 2
2.2 Updated Technology Stack Specifications . . . . . 3

**3 Detailed Implementation & Delivered Features** **3**
3.1 Artisan Business Intelligence & Analytics Engine . . . . . 3
3.2 Print-Ready Financial & Performance Statement Engine . . . . . 4
3.3 Conversational Commerce & WhatsApp Lead Attribution . . . . . 4
3.4 Multi-Tier Seller Subscription & Quota Enforcement . . . . . 4
3.5 Promotional Boosts & Automated Expiration Daemon . . . . . 5
3.6 Administrative Governance & Seller Malpractice Suspension . . . . . 5
3.7 Atomic Checkout & Order Fulfillment Workflow . . . . . 6
3.8 Multi-Entity Live Search Autocomplete & UI Modernization . . . . . 6

**4 Algorithmic Models & Formulations** **6**
4.1 Conversion Funnel & Business Intelligence Metrics . . . . . 6
4.2 Authoritative Boost Expiration Model . . . . . 7
4.3 Subscription Quota Validation Rule . . . . . 7

**5 Database Design & Entity Relationships** **7**

**6 Core Implementation Highlights** **8**
6.1 Automated Boost Synchronization & Quota Recovery . . . . . 8
6.2 Administrative Seller Suspension & Catalog Cascade . . . . . 8
6.3 Atomic Checkout with Concurrent Stock Verification . . . . . 9

**7 Challenges Solved & Technical Solutions** **10**

**8 Future Roadmap & Conclusion** **11**
8.1 Upcoming Engineering Milestones (Phase III) . . . . . 11
8.2 Conclusion . . . . . 11

**Abstract**

Following the successful implementation of the foundational Model-Template-View (MTV) architecture, three-tier Role-Based Access Control (RBAC), guest cart synchronization, and instant artisan publishing in Week 1, the **RuralConnect** project progressed into its commercialization, intelligence, and platform governance phase during the week of **November 02, 2026 to November 07, 2026**.

This technical report documents the implementation of major second-phase modules:

1. An **Artisan Business Intelligence & Analytics Engine** providing real-time time-series sales metrics, multi-stage customer conversion funnels, inventory warnings, and a formal print-ready financial statement/ledger (`analytics_report.html`).
2. A **Direct WhatsApp Conversational Commerce Pipeline** featuring automated contextual inquiry URL generation and background lead attribution tracking (`WhatsAppLead`, `ProductDailyView`).
3. A **Multi-Tier Seller Subscription & Quota Gating Engine** (`SellerSubscriptionPlan`, `SellerSubscription`) controlling product creation quotas.
4. A **Promotional Boost Lifecycle Engine** with self-healing background synchronization (`sync_expired_boosts`) and an automated CLI daemon (`expire_boosts.py`).
5. **Administrative Risk Governance** enabling instantaneous seller malpractice suspension cascading to active store listings.
6. An **Atomic Transaction-Safe Order Placement & Tracking Workflow** (`checkout_place_order_api`) with concurrent stock validation and multi-address support.
7. A **Sub-Second Multi-Entity Search Autocomplete Engine** (`search_suggest_api`) paired with a Glassmorphism UI redesign across all customer and management portals.

# 1 Introduction & Week 2 Objectives

## 1.1 Phase Review & Evolution

In the preceding project sprint (October 26 to October 31, 2026), **RuralConnect** established its baseline infrastructure: a custom user model supporting distinct roles (Buyer, Seller, Admin), an authoritative pricing engine protecting margins against conflicting discounts, unauthenticated guest session cart migration, dynamic wishlists, and OTP-based verification.

During the second sprint (November 02 to November 07, 2026), project development focused on transforming the baseline prototype into a fully transactional, financially viable, data-driven, and strictly moderated marketplace. Rural craftspeople often operate with low digital literacy and distinct operational constraints; bridging this gap required direct conversational channels, zero-friction financial accounting, time-bounded promotional tools, and robust platform safeguards against bad actors.

## 1.2 Primary Objectives for Week 2

1. **Artisan Business Intelligence & Financial Reporting:** Construct an end-to-end sales analytics pipeline delivering conversion funnels, revenue time-series graphs, and a printable ledger statement for rural micro-entrepreneurs.

2. **Conversational Commerce & Direct Lead Attribution:** Integrate direct-to-artisan WhatsApp inquiry capabilities with automatic price/product prefilling and asynchronous click attribution tracking.

3. **Seller Monetization & Quota Enforcement:** Introduce tiered subscription plans (Monthly/Annual) with enforced product listing limits on catalog CRUD operations.

4. **Self-Expiring Promotional Boosts:** Implement a promotional boost system with mathematical expiration safeguards and a cron-compatible Django management command.

5. **Administrative Risk Control & Seller Suspension:** Provide platform administrators with single-action powers to suspend rogue sellers, lock accounts, and batch-quarantine offending catalog items.

6. **Atomic Order Placement & Customer Fulfillment:** Build an atomic checkout API with database transactions to ensure race-condition-free inventory decrement and order generation.

7. **Multi-Entity Live Search & Modern Glassmorphic UI:** Develop a fast autocomplete endpoint matching products, categories, and artisans, complemented by modern responsive Glassmorphism styling.

# 2 System Architecture & Module Additions

## 2.1 Modular Architecture Progression

The modular Django architecture was extended to accommodate monetization, tracking, and governance workflows without tightly coupling domain logic:

- `seller`: Expanded beyond basic CRUD to incorporate analytical aggregation queries, subscription validation checks, printable financial statements, and order stage transitions.

- `marketplace`: Enhanced with transaction-wrapped checkout endpoints, asynchronous WhatsApp lead logging, daily impression counters, live search suggestion APIs, and boost synchronization routines.

- `platform_admin`: Upgraded with seller suspension controllers, boost revenue analytics, campaign scheduling engines, and multi-parameter catalog moderation tables.

- `accounts`: Enhanced with subscription validity helper methods (`has_active_subscription`), international phone formatting for WhatsApp communication, and delivery address default resolution.

| Layer | Technology | Week 2 Implementations & Functional Role |
|---|---|---|
| Backend Logic | Python 3.12, Django 5.x | Atomic transactions (`transaction.atomic`), ORM aggregations (`TruncDate`, `Sum`, `Avg`), management commands. |
| Database | PostgreSQL / SQLite | Compound indexing on daily views and leads; foreign-key cascading across subscription records. |
| Analytics Engine | Django ORM + Chart.js JSON | Time-series aggregation across 7d/30d/90d/year; conversion funnel computation; sales velocity calculation. |
| Conversational API | WhatsApp Click-to-Chat | Context-aware wa.me URL builder with international phone parsing and async lead tracking. |
| Monetization | Django Model Logic | Tier-based listing quota checks, automated expiration daemons, subscription gating decorators. |
| Frontend & UI | Glassmorphism CSS3 | Redesigned components, responsive drawer filters, sticky action bars, print media stylesheets. |
| Client-Side Scripts | Vanilla ES6+ JavaScript | Live search autocomplete, debounced fetch calls, toast micro-notifications, AJAX lead logging. |

Table 1: Incremental Technology Stack Specifications for Week 2

### 2.2 Updated Technology Stack Specifications

## 3 Detailed Implementation & Delivered Features

### 3.1 Artisan Business Intelligence & Analytics Engine

Rural artisans require transparent visibility into their commercial performance without having to navigate convoluted enterprise software. During this sprint, a comprehensive analytics suite was deployed in `seller/views.py` and rendered in `templates/seller/analytics.html`:

- **Core Executive Metrics:** Dynamic calculation of Gross Revenue, Total Units Sold, Total Orders Processed, Average Order Value (AOV), and Return/Cancellation Revenue.
- **Customer Conversion Funnel:** Quantifies buyer drop-off across four explicit operational stages:

$$\begin{aligned} \text{Catalog Impressions} &\longrightarrow \text{Customer Engagement (Leads + Wishlists)} \\ &\longrightarrow \text{Orders Placed} \longrightarrow \text{Delivered Orders} \end{aligned} \tag{1}$$

- **Time-Series Revenue & Traffic Trends:** Date-truncated aggregation (`TruncDate`) feeding serializable JSON arrays (`chart_revenue_json`, `chart_views_json`) across variable time horizons (7 Days, 30 Days, 90 Days, Year-to-Date, and All-Time).

- **Regional Provenance Mapping:** Aggregates shipping state information from customer delivery addresses to provide artisans with an empirical geographic breakdown of customer demand across India.
- **Automated Business Insights:** Heuristic rule cards providing instant guidance (e.g., flagging low-stock craft lines, highlighting items with high wishlist demand but zero inventory, and nudging prompt order fulfillment).

## 3.2 Print-Ready Financial & Performance Statement Engine

To empower rural craftspeople who interface with local microfinance institutions, artisan cooperatives, or tax authorities, a dedicated printable financial statement view (`analytics_report`) was constructed:

- Formats all financial figures into a formal, stamp-ready ledger document with dedicated print CSS media queries.
- Summarizes net realizable revenue, payment method distribution (UPI vs Cash-on-Delivery), and delivered vs pending consignments.
- Provides an itemized transaction ledger recording customer details, dates, unit quantities, and individual order fulfillment statuses.

## 3.3 Conversational Commerce & WhatsApp Lead Attribution

A significant barrier for rural artisans is the complexity of digital communication portals. Integrating WhatsApp provides a natural, high-trust communication channel while maintaining platform data attribution:

- **Contextual URL Generation (`Product.get_whatsapp_inquiry_url`):** Formats the artisan's registered contact into an E.164 international standard number (e.g., `+91...`) and builds a prefilled inquiry containing the product title, final discounted price, and direct product URL.
- **Asynchronous Click-Tracking Service (`track_whatsapp_click`):** When a buyer taps the "Chat with Artisan" button, an asynchronous AJAX request records a `WhatsAppLead` record before opening WhatsApp, capturing buyer identity, seller profile, product ID, and timestamp.
- **Daily View Attribution (`ProductDailyView`):** Tracks daily catalog impressions per product and seller with compound database indexes for sub-millisecond query performance during analytics aggregation.

## 3.4 Multi-Tier Seller Subscription & Quota Enforcement

To establish sustainable platform monetization, a subscription engine was implemented in `seller/models.py` and integrated into catalog management:

- **Subscription Model Architecture:** Introduces `SellerSubscriptionPlan` and `SellerSubscription`. Plans define duration, price, listing limits (`max_products`), and promotional perks.

- **Catalog Creation Gating:** The `add_product` and `edit_product` views inspect active seller subscriptions via `seller.has_active_subscription()`. Unsubscribed sellers are automatically redirected to plan selection with a preservation query parameter (`?next=...`).
- **Listing Quota Enforcement:** For tiers with listing constraints (e.g., starter plans), the system verifies current catalog depth against `plan.max_products`, prompting artisans to upgrade to the *Heritage Annual Pass* for unlimited inventory slots.

## 3.5 Promotional Boosts & Automated Expiration Daemon

To provide verified artisans with heightened storefront visibility without distorting marketplace fairness, a time-bounded promotional boost engine was implemented:

- **Boost Lifecycle Management:** Artisans purchase promotional boosts (`BoostPlan`) that elevate their crafts to sponsored positions on the homepage, deal carousels, and search results.
- **Self-Healing Expiration Routine (`sync_expired_boosts`):** Automatically transitions elapsed boosts to `STATUS_EXPIRED`, audits affected products, and resets `Product.promoted = False` unless a concurrent active boost exists.
- **Automated Django Management Command (`expire_boosts.py`):** An executable command suitable for crontab scheduling that executes the synchronization routine and outputs summary metrics to standard output.
- **Balanced Feed Sampling:** Homepage curation algorithms randomize and interleave boosted products with organic new arrivals, preventing sponsored listings from monopolizing catalog discovery.

## 3.6 Administrative Governance & Seller Malpractice Suspension

To maintain craft authenticity and protect buyers against fraud or non-fulfillment, platform governance was significantly strengthened in `platform_admin/views.py`:

- **One-Click Malpractice Suspension (`toggle_seller_suspension`):** Platform administrators can instantly suspend a seller profile. The action deactivates the user account (`user.is_active = False`), instantly terminates their active session via auth decorators, and executes a cascading batch-update marking all their live listings as `STATUS_SUSPENDED`.
- **Safe Restoration Protocol:** When an administrative audit clears a seller, the restore action re-enables the user account and safely restores suspended listings back to `STATUS_APPROVED`.
- **Granular Catalog Moderation:** The `manage_products` console was upgraded with multi-parameter filtering: moderation status, promotional status, category taxonomies, stock-out detection, and price sorting.

### 3.7 Atomic Checkout & Order Fulfillment Workflow

During Week 2, the end-to-end purchasing lifecycle was completed from shopping cart to order delivery:

- **Atomic Order Placement (`checkout_place_order_api`):** Wrapped in `transaction.atomic()`, the endpoint validates real-time stock levels, applies authoritative pricing rules, calculates shipping, creates parent `Order` and individual `OrderItem` records, decrements product inventory, and empties the customer's cart in a single atomic transaction.
- **Multi-Address Selection:** Integrates with the `DeliveryAddress` model, allowing buyers to select existing home/office addresses or supply inline delivery details.
- **Order Tracking & Fulfillment Portals:** Customers monitor order status stages (*Pending*, *Confirmed*, *Shipped*, *Delivered*) on `my_orders.html`, while artisans update dispatch timelines directly from their order management dashboard.

### 3.8 Multi-Entity Live Search Autocomplete & UI Modernization

To accelerate craft discovery and elevate user engagement, the storefront frontend received major performance and aesthetic enhancements:

- **Multi-Entity Autocomplete API (`search_suggest_api`):** A debounced, lightweight JSON endpoint returning instantaneous tri-facet search suggestions: matching products (with thumbnails and final prices), craft categories (with emojis/slugs), and verified artisan studios (with geographic locations).
- **Glassmorphism Design Overhaul:** Deployed unified CSS design tokens across `marketplace.css`, `dashboard.css`, and `responsive.css`, featuring translucent glass backdrops, mobile-optimized filter drawers, sticky mobile checkout actions, and micro-interaction toast notifications.

## 4 Algorithmic Models & Formulations

### 4.1 Conversion Funnel & Business Intelligence Metrics

The seller analytics engine computes the customer conversion rate ($CR$) and Average Order Value ($AOV$) over any selected time window $T$:

$$CR = \left( \frac{N_{\text{orders}}}{N_{\text{views}}} \right) \times 100\% \tag{2}$$

$$AOV = \frac{\sum_{k=1}^{N_{\text{items}}} (P_{k,\text{final}} \times Q_k)}{N_{\text{orders}}} \tag{3}$$

where $N_{\text{orders}}$ is the total number of distinct orders, $N_{\text{views}}$ represents aggregate page impressions from `ProductDailyView`, $P_{k,\text{final}}$ is the line item unit price, and $Q_k$ is the unit quantity.

### 4.2 Authoritative Boost Expiration Model

To guarantee that products are only marked promoted when backed by a paid, non-expired boost, the state of the product promotion flag is defined by:

$$\text{Promoted}(p) = \begin{cases} \text{True}, & \exists\, b \in \mathcal{B}_p : (b.\text{status} = \text{active} \wedge b.t_{\text{start}} \leq t \leq b.t_{\text{end}}) \\ \text{False}, & \text{otherwise} \end{cases} \tag{4}$$

where $\mathcal{B}_p$ is the set of all boosts associated with product $p$, and $t$ is the current system date.

### 4.3 Subscription Quota Validation Rule

Product listing authorization is governed by the following conditional rule:

$$\begin{aligned} \text{CanPublish}(s) = \text{HasValidSub}(s) \wedge \\ (\text{Plan}(s).\text{max_products} = 0 \vee |\mathcal{P}_s| < \text{Plan}(s).\text{max_products}) \end{aligned} \tag{5}$$

where $s$ is the artisan profile, $\text{HasValidSub}(s) \iff \exists$ active subscription with $t \leq$ end_date, and $|\mathcal{P}_s|$ represents the artisan's current catalog size.

## 5 Database Design & Entity Relationships

During Week 2, the relational schema was expanded with four key models to support subscriptions, traffic logging, and conversational leads:

| **Model Entity** | **Relationship Mapping** | **Core Purpose & Constraints** |
|---|---|---|
| `SellerSubscriptionPlan` | 1 ↔ Many with `SellerSubscription` | Defines commercial tiers (Monthly, Annual), pricing, duration in days, and product listing quota limits. |
| `SellerSubscription` | Many ↔ 1 with `SellerProfile` | Tracks artisan billing records, active status, expiration timestamps, payment tokens, and auto-renewal flags. |
| `ProductDailyView` | Many ↔ 1 with `Product` & `SellerProfile` | Aggregates daily catalog impression counts. Indexed compoundly on `(seller, date)` and `(product, date)`. |
| `WhatsAppLead` | Many ↔ 1 with `Product`, `SellerProfile`, & `User` | Logs buyer click-to-chat inquiry events for lead attribution and artisan engagement tracking. |
| `Boost` | Many ↔ 1 with `BoostPlan` & `Product` | Governs sponsored product visibility periods; managed by custom `BoostQuerySet`. |

Table 2: New and Extended Database Entities in Week 2

# 6 Core Implementation Highlights

## 6.1 Automated Boost Synchronization & Quota Recovery

```python
def sync_expired_boosts():
    """
    Finds active boosts whose end_date has passed, marks them
    as 'expired', and resets Product.promoted=False if the product
    has no other live boost. Cleans orphaned promoted flags.
    """
    today = timezone.now().date()
    past_active_boosts = Boost.objects.filter(
        status=Boost.STATUS_ACTIVE,
        end_date__lt=today
    )
    affected_product_ids = set(
        past_active_boosts.values_list('product_id', flat=True)
    )

    if past_active_boosts.exists():
        past_active_boosts.update(status=Boost.STATUS_EXPIRED)

    for pid in affected_product_ids:
        has_active = Boost.objects.filter(
            product_id=pid,
            status=Boost.STATUS_ACTIVE,
            start_date__lte=today,
            end_date__gte=today
        ).exists()
        if not has_active:
            Product.objects.filter(pk=pid).update(promoted=False)

    valid_promoted_pids = Boost.objects.filter(
        status=Boost.STATUS_ACTIVE,
        start_date__lte=today,
        end_date__gte=today
    ).values_list('product_id', flat=True)
    Product.objects.filter(promoted=True).exclude(
        id__in=valid_promoted_pids
    ).update(promoted=False)
```

Listing 1: Authoritative Boost Expiration Service

## 6.2 Administrative Seller Suspension & Catalog Cascade

```python
@admin_required
def toggle_seller_suspension(request, pk):
    seller = get_object_or_404(SellerProfile, pk=pk)
    if request.method == 'POST':
        action = request.POST.get('action')
        if action == 'suspend':
            seller.user.is_active = False
            seller.user.save(update_fields=['is_active'])
            affected = Product.objects.filter(seller=seller)\
                .exclude(status=Product.STATUS_REJECTED)\
```

```
                .update(status=Product.STATUS_SUSPENDED)
            messages.warning(
                request,
                f'Seller "{seller.business_name}" suspended. '
                f'Account locked and {affected} listings removed from store.'
            )
        elif action == 'restore':
            seller.user.is_active = True
            seller.user.save(update_fields=['is_active'])
            affected = Product.objects.filter(
                seller=seller,
                status=Product.STATUS_SUSPENDED
            ).update(status=Product.STATUS_APPROVED)
            messages.success(
                request,
                f'Seller "{seller.business_name}" restored. '
                f'{affected} listings reactivated.'
            )
    return redirect(f"{reverse('platform_admin:dashboard')}?open_sellers=1")
```

Listing 2: Seller Suspension Governance Action

## 6.3 Atomic Checkout with Concurrent Stock Verification

```
@require_POST
def checkout_place_order_api(request):
    cart_items = CartItem.objects.filter(user=request.user).select_related('product')
    if not cart_items.exists():
        return JsonResponse({'success': False, 'error': 'Cart is empty'}, status=400)

    with transaction.atomic():
        # Validate stock availability across all line items
        for item in cart_items:
            if item.product.stock < item.quantity:
                return JsonResponse({
                    'success': False,
                    'error': f'Insufficient stock for {item.product.name}'
                }, status=400)

        # Create parent order
        order = Order.objects.create(
            user=request.user,
            shipping_address=selected_address,
            payment_method=payment_method,
            total_amount=calculated_total
        )

        # Generate line items and decrement inventory atomically
        for item in cart_items:
            OrderItem.objects.create(
                order=order,
                product=item.product,
```

```
                seller=item.product.seller,
                quantity=item.quantity,
                unit_price=item.product.final_price(),
                line_total=item.product.final_price() * item.quantity
            )
            item.product.stock -= item.quantity
            item.product.save(update_fields=['stock'])

        cart_items.delete()

    return JsonResponse({'success': True, 'order_id': order.id})
```

Listing 3: Atomic Order Placement Pipeline

# 7 Challenges Solved & Technical Solutions

- **Challenge 1: Bridging Digital Literacy with WhatsApp Direct Commerce**
  *Problem:* Rural craftspeople frequently abandon complex digital marketplaces because managing web-based customer inquiries is intimidating or technically prohibitive.
  *Solution:* Engineered a direct-to-artisan WhatsApp pipeline that generates contextual, pre-filled inquiry links in the customer's mobile app while asynchronously registering lead records (`WhatsAppLead`) on the backend for sales attribution.

- **Challenge 2: Preventing Promotional State Inconsistencies & Orphaned Boosts**
  *Problem:* If an artisan's promotional boost expires at midnight, cached flags or server downtime could cause non-paying products to remain featured indefinitely.
  *Solution:* Formulated a deterministic synchronization engine (`sync_expired_boosts`) that evaluates active boosts on demand, resets stale product flags, and operates concurrently via a scheduled CLI command (`expire_boosts.py`).

- **Challenge 3: Seamless Subscription Gate Redirection**
  *Problem:* Gating product creation behind a paid subscription can frustrate artisans if they fill out a complex listing form only to be blocked upon submission.
  *Solution:* Implemented proactive view gates on both `add_product` and `edit_product` that detect missing or expired subscriptions upon initial entry, redirecting artisans to the subscription tier page with an automatic `?next=` parameter that resumes listing creation once subscribed.

- **Challenge 4: Race Conditions in Order Inventory Decrement**
  *Problem:* Concurrent checkouts on limited-quantity handcrafted items risked selling inventory below zero.
  *Solution:* Enclosed order creation, multi-item stock validation, line item generation, and inventory decrements inside a database transaction block (`transaction.atomic()`) with strict pre-commit verification.

# 8 Future Roadmap & Conclusion

## 8.1 Upcoming Engineering Milestones (Phase III)

- **Payment Gateway Webhook Integration:** Replacing simulated checkouts with live Razorpay/Stripe webhooks and UPI deep-linking for seamless mobile payments.
- **Automated SMS & WhatsApp Dispatch Alerts:** Integrating Twilio or Gupshup SMS webhooks to notify artisans of incoming orders via basic text messaging without requiring smartphone data connectivity.
- **Multilingual Audio Descriptions:** Introducing audio snippet generation for artisans who wish to narrate the cultural heritage and craftsmanship behind their artifacts in regional vernaculars.

## 8.2 Conclusion

During the second sprint (November 02 to November 07, 2026), **RuralConnect** successfully evolved from an architectural prototype into a commercial-grade, multi-role e-commerce platform. The introduction of artisan business analytics, formal print-ready ledgers, WhatsApp lead attribution, subscription tier enforcement, automated promotional lifecycles, administrative malpractice controls, and atomic order checkouts establishes a resilient and equitable digital marketplace that honors both artisan simplicity and platform integrity.